

راهنمای سه سطحی موتورهای تطبیقی آزمونک (آزمونک)

الگوریتم‌های آزمونک در سه لایه

یک الگوریتم، سه ارتفاع: از توضیح بچه ۱۰ ساله تا کد آماده پایتون

لایه کودکانه



قصه و تشبیه – اگر هیچ ریاضی‌ای بلد نباشی هم می‌فهمی

لایه مفهومی



ایده، فرمول و مثال عددی گام به گام – مغز الگوریتم

لایه مهندسی



کد پایتون واقعی و تست‌شده – آماده کپی به هسته پروژه

پدیدآور

پویا – جشنواره نوجوان خوارزمی

رشته رایانه / نرم‌افزار / هوش مصنوعی

فهرست

هر فصل یک موتور است و هر موتور در سه لایه باز می‌شود؛ ترتیب خواندن پیشنهادی، همان ترتیب این فهرست است

راهنمای خواندن – سه لایه در دو دقیقه	۰
موتور Elo – امتیاز هوشمند	۱
عمیق‌ترین فصل؛ قلب تپنده سایت	
جایابی – کجا شروع کنی؟	۲
۴ سؤال، ۳ رده، یک عدد	
انتخاب سؤال – ناحیه طلایی	۳
سه فیلتر: هدف، ارزش، مواجهه	
نردبان سطح D16 – لیگ‌های هر دسته	۴
۲ غلط ← سقوط، ۲ درست ← ارتقا	
SM-2 – مرور فاصله‌دار	۵
۱ ← ۶ ← ۱۵ روز؛ قبل از فراموشی	
پیش‌بینی نمره	۶
نمره وزن‌دار + رگرسیون λ	
قواعد اجرا – قانون‌های بازی	۷
جلسه، اعتماد، بنر مطالعه	
همه‌چیز در یک نگاه	۸
چرخه + جدول تصمیم‌های D1 تا D16	
پیوست الف – دستگیره‌های قابل تنظیم	الف
جدول کامل پارامترها	
پیوست ب – واژه‌نامه و اجرای مازول‌ها	ب
فارسی ↔ English + تست سریع	

💡 این سند نسخه سه‌لایه است و کنار «الگوریتم‌های آزمونک» (نسخه تخصصی ۳۱ صفحه‌ای) می‌نشیند؛ آن‌جا استخراج کامل ریاضی و مقایسه‌های مفصل آمده و این‌جا هدف «فهم واقعی» است.

سه لایه در دو دقیقه

قبل از شروع، یاد بگیر چطور از این کتاب استفاده کنی

هر موتور در آزمونک یک تصمیم مهندسی پشتش دارد: «چند امتیاز بده یا بگیر؟»، «کدام سؤال را سرو کنم؟»، «کی مرور کند؟». این کتاب هر موتور را در **سه پله** توضیح می‌دهد و ترتیب پله‌ها در همه فصل‌ها یکسان است. می‌توانی هر سه پله را پشت‌سرهم بخوانی، یا فقط از پله موردنیازت شروع کنی و بالاترها را بعداً برداری.

 **لایهٔ کودکانه** پلهٔ اول – انگار داری به یک بچهٔ ۱۰ ساله توضیح می‌دهی

اینجا هیچ فرمولی نیست؛ فقط **قصه و تشبیه**. اگر این پله را بخوانی، «ایدهٔ اصلی» را همیشه در ذهنت داری – حتی وسط شب قبل از داوری. اگر روزی فراموش کردی الگوریتم چی بود، به همین قصه برگرد.

 **لایهٔ مفهومی** پلهٔ دوم – مغز الگوریتم: ایده، فرمول، مثال عددی

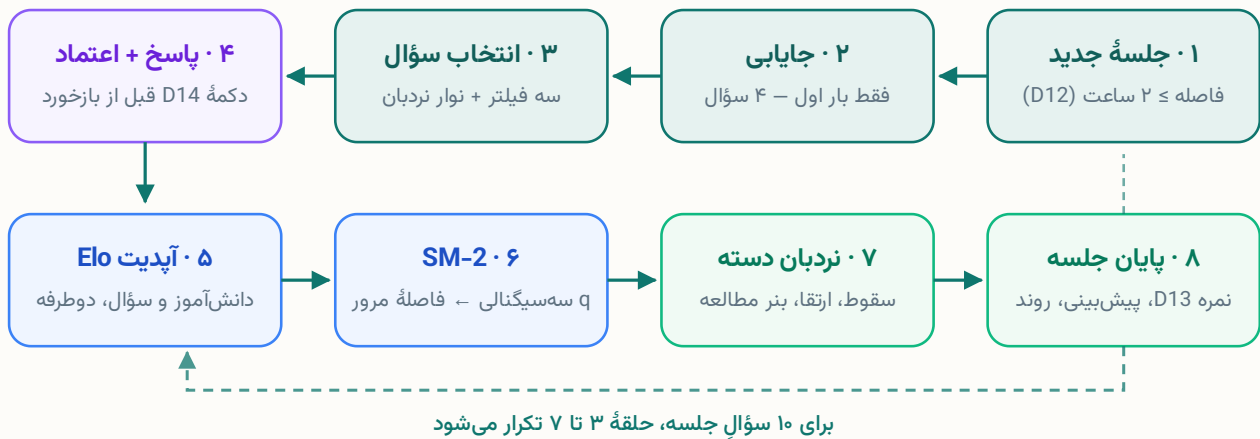
اینجا فرمول‌ها با **همهٔ نمادهایشان** معرفی می‌شوند و هر فرمول با یک مثال عددی گام‌به‌گام دنبال می‌شود که همهٔ اعدادش با کد واقعی تست شده‌اند. این پله همان چیزی است که در جلسهٔ داوری باید از زبانت روان بیرون بیاید.

 **لایهٔ مهندسی** پلهٔ سوم – کد پایتون واقعی، تست‌شده و آمادهٔ کپی به core

هر فصل یک فایل واقعی دارد (**elo.py**، **sm2.py** و...) که همین الان اجرا شده و تست‌هایش پاس شده‌اند. این فایل‌ها در پوشهٔ **core/** کنار همین PDF هستند؛ می‌توانی بازشان کنی، مقادیر را عوض کنی و ببینی نتیجه چه می‌شود – بهترین راه فهمیدن عمیق.

چرخهٔ کامل یک جلسهٔ آزمونک

هرچه در این کتاب می‌خوانی، در قلب همین چرخه می‌چرخد. بار اول که وارد می‌شوی، جایابی (گام ۲) اجرا می‌شود؛ از جلسهٔ بعد مستقیم به انتخاب سؤال می‌روی. پایان هر جلسه هم نمرهٔ وزن‌دار، پیش‌بینی و نمودار روند تولید می‌شود.



شکل ۱ - چرخه پاسخ آزمونک؛ رنگ هر گودال نشان می‌دهد کدام فصل آن را باز می‌کند

سه مسیر پیشنهادی: ① اگر وقت کم داری، فقط لایه‌های سبز و مثال‌ها را بخوان (حدود ۴۵ دقیقه). ② اگر می‌خواهی برای داوری آماده شوی، سبز + آبی کامل. ③ اگر می‌خواهی خودت الگوریتم را تغییر بدهی، بنفش‌ها را با فایل‌های core/ باز کنار دستت بخوان و مقادیر را دستکاری کن.

فصل ۱ - عمیق‌ترین فصل

موتور Elo - امتیاز هوشمند

قلب تپنده آزمونک: بعد از هر پاسخ، چه کسی چند امتیاز بگیرد یا بدهد؟

در یک آزمون کاغذی، نمره ۱۸ از ۲۰ فقط یک جمله می‌گوید: «این فصل را خوب بلدی». اما همان نمره اگر از سؤال‌های سخت گرفته شده باشد معنای دیگری دارد تا وقتی از سؤال‌های آسان است. آزمونک می‌خواهد همین تفاوت را **عددی و لحظه‌به‌لحظه** بسازد؛ پس به جیبی نیاز دارد که بعد از هر تک‌پاسخ، هم دانش‌آموز و هم سؤال را دوباره ارزیابی کند. آن جیب Elo است - همان موتوری که رتبه بازیکنان شطرنج جهان را چند دهه است مدیریت می‌کند.

اگر یک بچه ۱۰ ساله بپرسد: «این Elo دیگر چیست؟»

فرض کن در یک بازی آنلاین رتبه داری؛ مثل لیگ برنز و نقره و طلا. اگر با کسی هم‌سطح خودت بازی کنی و ببری، رتبه‌ات فقط کمی بالا می‌رود – چون کاری که انتظارش می‌رفت را کردی. ولی اگر با بازیکن خیلی قوی‌تری بازی کنی و او را ببری، رتبه‌ات جهش می‌کند؛ چون کاری کردی که هیچ‌کس فکرش را نمی‌کرد! و اگر برعکس از بچه‌ای کوچک‌تر ببازی، بردن تو هیچ‌کس را خوشحال نمی‌کند و رتبه‌ات تقریباً ثابت می‌ماند؛ ولی اگر ببازی، خیلی رتبه از دست می‌دهی.

آزمونک هم دقیقاً همین کار را می‌کند: **هر سؤال مثل یک حریف است.** سؤالی که نمره‌اش ۱۳۵۰ است مثل یک بازیکن قوی است؛ اگر جوابش بدهی، از او امتیاز می‌گیری. جالب‌تر اینکه خود سؤال هم رتبه دارد و آن رتبه با پاسخ بچه‌ها بالا و پایین می‌شود؛ اگر بچه‌های ضعیف‌تر از حد انتظار به یک سؤال جواب درست بدهند، یعنی آن سؤال «باحیان» آسان‌تر است و رتبه‌اش کمی پایین می‌آید. مثل این است که همه کلاس به حریف امتیاز بدهند یا بگیرند!

مفهوم – دو عدد، یک انتظار، یک



سورپرایز

هر دانش‌آموز یک «توان» و هر سؤال یک «سختی» روی یک خط‌کش مشترک

کل موتور سه جمله است. **اول:** از اختلاف دو عدد، «انتظار» ساخته می‌شود – شانس‌ی که دانش‌آموز دارد سؤال را درست جواب بدهد:

$$E = 1 / (1 + 10^{(R_q - R_s) / 400})$$

فرمول انتظار – منحنی لجستیک؛ ۴۰۰ ثابت مقیاس است

E	انتظار موفقیت؛ عددی بین ۰ و ۱. اگر توان دانش‌آموز با سختی سؤال برابر باشد، E دقیقاً ۰/۵ می‌شود
R _s	توان دانش‌آموز – شروع ۱۲۵۰ از جلسه جابایی
R _q	سختی سؤال – نویسنده برآورد اولیه می‌زند و «رأی جامعه پاسخ‌دهندگان» آن را اصلاح می‌کند
S	نتیجه واقعی: پاسخ درست = ۱، غلط = ۰
K	ضریب سرعت یادگیری؛ بزرگ نبودنش مهم است – جلوتر می‌بینیم چرا

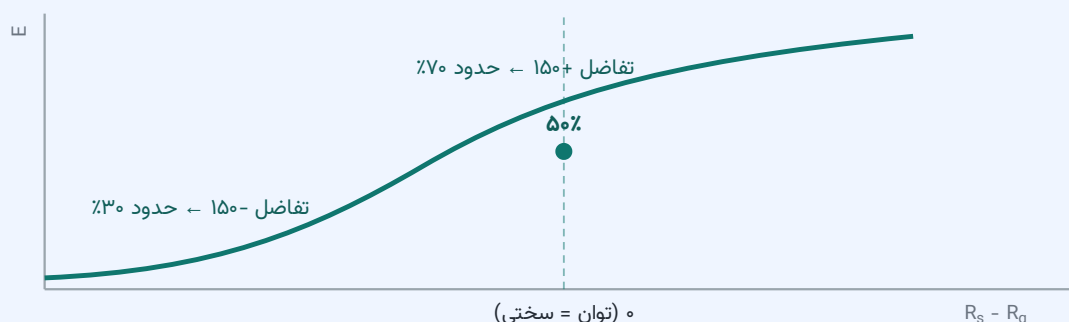
دوم: سورپرایز اندازه می‌شود؛ یعنی فاصله «انتظار» تا «واقعیت»: S - E. پاسخ درست به سؤالی که ۷۰٪ شانس درست‌زدنش را داشتی تقریباً هیچ خبری نیست؛ همین پاسخ به سؤالی با ۳۰٪ شانس یک خبر بزرگ است. **سوم:** هر دو طرف بر اساس همان سورپرایز جابه‌جا می‌شوند – آینه‌وار:

$$R_s' = R_s + K_s \cdot (S - E) \quad R_q' = R_q + K_q \cdot (E - S)$$

آپدیت دوطرفه – آن‌که بهتر از انتظار عمل کند می‌گیرد و دیگری همان را می‌دهد

دقت کن امضای دو فرمول برعکس است: اگر دانش‌آموز از انتظارش بهتر عمل کند ($S > E$) توانش بالا می‌رود و سختی سؤال پایین می‌آید. ضریب‌های K هم برابر نیستند و نباید باشند: هر پاسخ درباره «این دانش‌آموز» اطلاعات خیلی زیادی دارد، ولی درباره «این سؤال» فقط یک رأی از دل هزاران رأی است؛ پس $K_s = ۳۲$ ولی $K_q = ۸$. و هر دو با تجربه آرام می‌گیرند تا دیر هنگام نلرزند:

K_s	۳۲ تا پاسخ پنجاهم $\leftarrow$ ۲۴ تا پاسخ صد و پنجاهم $\leftarrow$ ۱۶ بعد از آن
K_q	۸ تا پاسخ چهل و سؤال $\leftarrow$ ۴ بعد از آن (سؤال به سختی واقعی‌اش می‌رسد)



شکل ۲ - منحنی انتظار: هر ۴۰۰ امتیاز اختلاف، شانس را ده برابر می‌کند؛ نقطه میانی همان ۵۰٪ است

 **مهندسی - فایل [core/elo.py](#)** ۳۰ خط پایتون خالص؛ بدون هیچ کتابخانه‌ای

کل موتور Elo در یک فایل کوچک جا می‌شود. سه تابع دارد: محاسبه انتظار، ضرایب متغیر K و آپدیت دوطرفه. کامنت‌ها فارسی‌اند تا موقع خواندن گم نشوی؛ اسم توابع و متغیرها انگلیسی است (طبق D5 - کد انگلیسی، مستندات فارسی).

```

"""آزمونک - رتبه‌بندی دوطرفه دانش‌آموز و سؤال Elo موتور"""

K_STUDENT_START = 32    # دانش‌آموز در شروع K
K_QUESTION_START = 8    # سؤال در شروع K
D_SCALE = 400.0         # ثابت مقیاس لجستیک؛ هر ۴۰۰ امتیاز = شانس ۱۰ برابر

def expected_success(r_student: float, r_question: float) -> float:
    """Elo. شانس درست‌زدن دانش‌آموز روی سؤال - فرمول لجستیک"""
    return 1.0 / (1.0 + 10.0 ** ((r_question - r_student) / D_SCALE))

def k_student(total_answers: int) -> int:
    """متغیر دانش‌آموز: ۳۲ تا ۵۰ پاسخ، ۲۴ تا ۱۵۰، بعد ۱۶ K"""
    if total_answers < 50:
        return 32
    if total_answers < 150:
        return 24
    return 16

def k_question(total_answers: int) -> int:
    """متغیر سؤال: هر پاسخ درباره سؤال اطلاعات کمتری دارد K"""
    return K_QUESTION_START if total_answers < 40 else 4

```

```

def apply_result(
    r_student: float,
    r_question: float,
    correct: bool,
    k_s: int = K_STUDENT_START,
    k_q: int = K_QUESTION_START,
) -> tuple[float, float, float]:
    """دانش‌آموز:  $R' = R + K_s * (S - E)$  | سؤال:  $R' = R + K_q * (E - S)$ 

    Returns:
        (قبل از پاسخ E توان جدید دانش‌آموز، سختی جدید سؤال، انتظار)
    """
    e = expected_success(r_student, r_question)
    s = 1.0 if correct else 0.0
    new_student = r_student + k_s * (s - e)
    new_question = r_question + k_q * (e - s)
    return new_student, new_question, e

```

چرا تابع، سه‌تایی برمی‌گرداند؟ چون سرویس جلسه به E قبل از پاسخ هم نیاز دارد - برای انتخاب سؤال بعدی (فصل ۳) و ذخیره در params_snapshot (G7). یک محاسبه، سه مصرف.

مثال عددی گام به گام – همان «تست طلایی» سند اصلی

۱ دانش‌آموزی با توان 1200 به سؤالی با سختی 1350 می‌رسد. انتظار: $E = 1/(1+10^{(150/400)}) = 0.296 \approx 1/(1+2.371)$ فقط ۲۹٫۶٪ شانس درست‌زدن.

۲ او درست جواب می‌دهد! سورپرایز: $S - E = 1 - 0.296 = 0.704$ – خبر بزرگ، چون کاری بهتر از انتظار انجام شد.

۳ دانش‌آموز: ۱۲۲۲٫۵ – $1200 + 32 \times 0.704$ – جهشی حدود ۲۲ امتیازی؛ با K متغیر، دفعات بعد این جهش کوچک‌تر می‌شود.

۴ سؤال: ۱۳۴۴٫۴ – $1350 + 8 \times (0.296 - 1) = 1350 - 5.6$ – سؤال کمی آسان‌تر شد؛ این همان «رأی جامعه» بر سختی است. (هر دو عدد با تست خودکار بازتولید می‌شوند: python (scripts/test_core_modules.py

دستگیره	مقدار فعلی	اگر عوضش کنی چه می‌شود؟
D_SCALE	400	بزرگ‌تر ← منحنی ملایم‌تر؛ سورپرایزها کوچک‌تر و امتیازها آرام‌تر حرکت می‌کنند
K_STUDENT_START	32	بزرگ‌تر ← رتبه تندتر بالا/پایین می‌رود ولی پرنوسان‌تر؛ نوسان پس از هر جلسه بدتر می‌شود
K_QUESTION_START	8	بزرگ‌تر ← سختی سؤال‌ها سریع‌تر اصلاح می‌شود ولی یک بچه خوش‌شانس می‌تواند سختی را بیپچاند
K آستانه‌های کاهش	50 / 150	کم‌تر کردنشان ← رتبه زودتر «آرام» می‌گیرد؛ زیاد کردنشان ← رتبه دیرتر پایدار می‌شود

چرا Elo و نه IRT / Glicko-2 / TrueSkill؟ IRT (نظریه پاسخ به سؤال) جامع‌تر است و در توفل و PISA کار می‌کند، ولی برای کالیبره‌شدن به صدها پاسخ در هر سؤال نیاز دارد – بانک سؤال MVP ما ۵۰ تا ۱۵۰ سؤال است و این سرمایه وجود ندارد. Glicko-2 و TrueSkill «عدم قطعیت» را هم مدل می‌کنند ولی پیچیدگی‌شان برای موتوری که باید یک دانش‌آموز دبیرستانی بتواند توضیحش بدهد، هزینه است نه فایده. Elo با ۳۰ خط پایتون، شفاف و قابل دفاع است و اگر روزی داده انبوه شد، مهاجرت به IRT از روی همان دو ستون توان/سختی ممکن است.

فصل ۲

جایابی – کجا شروع کنی؟

۴ سؤال، ۳ رده سختی، یک عدد قابل اعتماد (تصمیم D1)

اولین روزی که وارد آزمونک می‌شوی، سیستم هیچ چیز درباره‌ات نمی‌داند. سه راه رایج وجود دارد: خوداظهاری («خودت بگو چه قدر بلدی»), پیش‌فرض ثابت (به همه ۱۲۰۰ بده) و پیش‌آزمون. هر سه رد شده‌اند: خوداظهاری سوگیری دارد، پیش‌فرض ثابت برای نصف بچه‌ها اشتباه است و پیش‌آزمون هزینه و استرس دارد. انتخاب شد

جلسه جایابی: خودِ موتور Elo با ۴ سؤال، سطح واقعیات را پیدا کند – همان کاری را بکند که در بقیه عمرش می‌کند، فقط با K بزرگ‌تر و ثابت.

اگر یک بچه ۱۰ ساله بپرسد: «چطور بازی از سطح من سر در می‌آورد؟»

مثل بازی‌ای که اولش از تو می‌پرسد «تا حالا بازی کردی؟» و بعد چند مرحله آزمایشی می‌دهد: آسانی می‌دهد، اگر راحت رد کردی سخت‌تر می‌دهد، اگر سختش گیر کردی برمی‌گردد عقب. آزمونک هم اول ۴ سؤال از سه پله مختلف می‌دهد – پله متوسط، پله آسان، پله سخت – و با جواب‌هایت می‌فهمد «قد» امتیازیت حدوداً چقدر است.

یک راز کوچک: ترتیب این سؤال‌ها **پراکنده** است، نه اول آسان و بعد سخت! چون اگر ترتیب منظم باشد، بچه‌ها الگو را می‌فهمند و شروع می‌کنند به حدس‌زدن سیستماتیک: «چون سؤال قبلی سخت بود، این یکی حتماً آسان است». پراکندگی این تقلب را می‌بندد.

مفهوم – جایابی، همان Elo است با پدال گاز

هیچ فرمول جدیدی این‌جا وجود ندارد؛ همان آپدیت دوطرفه فصل ۱ است با سه تفاوت حساب‌شده. **تفاوت اول:** نقطه شروع ۱۲۵۰ است – وسط ناحیه ممکن، بی‌طرف نسبت به هیچ‌کس. **تفاوت دوم:** سختی ۴ سؤال روی سه رده ۱۱۵۰ / ۱۲۵۰ / ۱۳۵۰ پخش می‌شود (سه سؤال اول) و سؤال چهارم از رده‌ای انتخاب می‌شود که به برآورد جاری نزدیک است؛ این‌طوری هر پاسخ، برآورد را واقعاً جابه‌جا می‌کند. **تفاوت سوم:** در طول این ۴ پاسخ، K ثابت روی ۳۲ می‌ماند (تصمیم D1) – می‌خواهیم سریع به سطح واقعی برسیم؛ «آرام‌گیری» K شروع جلسه بعد است. خروجی جایابی دو چیز است: توان اولیه R_S و یک **شاخص عدم قطعیت** که با هر پاسخ کوچک‌تر می‌شود؛ اگر دفعه‌های بعد شاهد پرش‌های عجیب بودیم، همین شاخص به سیستم می‌گوید هنوز مطمئن نیستیم.



```

"""بیدا کردن سطح واقعی دانش‌آموز تازه‌وارد - (D1) جایابی ۴ سوالی"""

from elo import apply_result

START_RATING = 1250.0 # برآورد خنثی قبل از اولین پاسخ
PLACEMENT_K = 32 # ثابت در طول جایابی K
QUESTION_K = 8 # سؤال در جایابی K
# سه رده سختی با «ترتیب پراکنده» - دنباله صعودی الگو می‌سازد و حدس می‌آورد
SPREAD = (1250.0, 1150.0, 1350.0)
BASE_UNCERTAINTY = 100.0 # شاخص عدم قطعیت پایه = نصف پراکندگی
UNCERTAINTY_DECAY = 15.0 # کاهش عدم قطعیت با هر پاسخ
MIN_UNCERTAINTY = 40.0

def next_question_rating(index: int, r_student: float) -> float:
    """اُم: سه سؤال اول بخش روی رده‌ها، چهارمی نزدیک برآورد سختی سؤال"""
    if index < len(SPREAD):
        return SPREAD[index]
    return min(1350.0, max(1150.0, r_student))

def run_placement(answers: list[bool]) -> dict:
    """اولیه + شاخص عدم قطعیت + ردیابی R_S: کل جلسه جایابی؛ خروجی"""
    r_student = START_RATING
    trace = []
    for i, correct in enumerate(answers):
        r_question = next_question_rating(i, r_student)
        new_student, new_question, e = apply_result(
            r_student, r_question, correct, PLACEMENT_K, QUESTION_K
        )
        trace.append({
            "step": i + 1, "r_question": r_question,
            "expected": round(e, 3), "correct": correct,
            "r_student": round(r_student, 1),
            "r_student_new": round(new_student, 1),
        })
        r_student = new_student
    uncertainty = max(
        MIN_UNCERTAINTY,
        BASE_UNCERTAINTY - UNCERTAINTY_DECAY * len(answers),
    )
    return {"r_student": round(r_student, 1),
            "uncertainty": uncertainty, "trace": trace}

```

مثال عددی گام به گام – جلسه جایابی با الگوی «درست، غلط، درست، درست»

- ۱ سؤال 1250 (رده میانی): $E = 0.500$ ؛ درست $\rightarrow 1250 + 32 \times 0.5 = 1266$.
- ۲ سؤال 1150 (پراکنده به عقب): $E = 0.661$ ؛ غلط $\rightarrow 1266 - 32 \times 0.661 = 1244.8$ – غلط «غیرمنتظره» سقوط می‌آورد.
- ۳ سؤال 1350: $E = 0.353$ ؛ درست $\rightarrow 1244.8 + 32 \times 0.647 = 1265.5$ – جهش زیر فشار.
- ۴ سؤال چهارم از نزدیک‌ترین رده به برآورد: 1266؛ $E = 0.5$ ؛ درست $\rightarrow 1265.5 + 32 \times 0.5 = 1281.5$. خروجی: توان اولیه $1281.5 + ۱۲۸۱/۵$ عدم قطعیت ۴۰.

چرا جایابی «کوچک» و نه آزمون تطبیقی کامل (CAT)؟ آزمون تطبیقی استاندارد، هر سؤال را با بهینه‌سازی آماری برمی‌گزیند و به ده‌ها سؤال و بانک کالیبره نیاز دارد. جایابی ۴ سؤالی، ۸۰٪ فایده را با ۵٪ هزینه می‌گیرد و چون از همان موتور Elo استفاده می‌کند، هیچ گذار و ناسازگاری بعد از جلسه اول وجود ندارد؛ کد هم یک حلقه ۴ خطی است.

فصل ۳

انتخاب سؤال – ناحیه طلایی

سه فیلتر پشت‌سرهم: چقدر سخت؟ چقدر مفید؟ آیا اخیراً دیده‌ای؟

حالا که موتور «می‌داند» توان تو چیست، باید تصمیم بگیرد بعدی چه سؤالی سرو کند. این تصمیم مهم‌ترین تجربه کاربری کل محصول است: اگر سؤال‌ها همیشه آسان باشند، اپ را با حس «وقت تلف کردن» ترک می‌کنی و اگر همیشه سخت باشند، با حس «من نمی‌توانم». سیاست سند، **ناحیه تقریبی توسعه** است: جایی که کار سخت است اما نه به قدری که ناامیدکننده باشد. این سیاست در عمل سه لایه پشت‌سرهم دارد که از کلان به خرد اجرا می‌شوند.

اگر یک بچه ۱۰ ساله بپرسد: «چرا این سؤال‌ها را برای من می‌آورد؟»

تو بازی کرده‌ای: وقتی همه مراحل خیلی آسان باشند، پنج دقیقه‌ای حوصله‌ات سر می‌رود؛ وقتی همه خیلی سخت باشند، گوشی را پرت می‌کنی رفتی. بهترین بخش بازی آن‌جاست که **هفت‌هشت‌بار از ده می‌توانی** – سخت، ولی نه غیرممکن. آزمونک هم دنبال همان حس است: سؤالی که حدود ۷۰٪ دفعه‌ها می‌توانی جوابش بدهی.

بعدش هم مثل مربی باهوش عمل می‌کند: بین چند سؤال «در حد تو»، آن‌که به **ضعف تو** مربوط است را اول می‌دهد، آن‌که خیلی وقت است ندیده‌ای را یادت می‌اندازد، و سؤالی که دیروز ده بار دیده‌ای را دوباره جلوتر نمی‌آورد – چون حفظ کردن صورت سؤال، یادگیری نیست!

لایهٔ اول – ناحیهٔ هدف ۶۵٪ تا ۷۵٪. فرمول انتظار فصل ۱ برای هر سؤالِ کاندید یک عدد می‌دهد؛ سیاست، سؤالی است که این شانس داخل بازه باشد و عملیاتی‌کردنش یعنی انتخابِ کمینهٔ فاصله از میانه:

$$q^* = \operatorname{argmin}_{q \in \text{Candidates}} |E(R_s, R_q) - 0.70|$$

معیار کمینهٔ فاصله از نقطهٔ هدف ۰٫۷۰ – از اسلایدهای سند

چرا ۷۰٪ و نه ۵۰٪؟ در ۵۰٪ عملاً با شانس سکه جلو می‌روی؛ اضطراب بالا می‌رود و بازخورد کم اطلاع می‌شود. چرا نه ۹۰٪؟ چون آن‌جا دیگر «چالش» نیست؛ نمرهٔ بالا فقط خودفریبیِ آسان‌گرایی می‌سازد. ۷۰٪ نقطهٔ تعادل حسِ پیشرفت و چالش واقعی است و بازهٔ ۶۵–۷۵٪ هم اجازهٔ تنفس می‌دهد. نکتهٔ جالب ریاضی: چون ۷۰٪ یعنی سؤال باید «کمی آسان‌تر از تو» باشد، این ناحیه معمولاً کمی پایین‌تر از R_s تو قرار می‌گیرد – این باگ نیست، جنسِ تعریف E است.

لایهٔ دوم – امتیاز ارزش آموزشی. معمولاً چند سؤال هم‌زمان داخل ناحیه‌اند؛ این‌جا سیاست از «سخت کدام» به «مفید کدام» سوییچ می‌کند. هر کاندید جمع وزنی سه مؤلفه می‌گیرد:

$$V(q) = w_1 \cdot \text{Weakness}(\text{cat}) + w_2 \cdot \text{Importance}(\text{topic}) + w_3 \cdot \text{Recency}(q)$$

وزن‌ها ۰٫۵ / ۰٫۳ / ۰٫۲ – پیش‌فرض مهندسی و قابل و تو

Weakness از نردبان سطح (فصل بعد) می‌آید: دسته‌ای که سطحش افتاده امتیاز بیشتری می‌گیرد. **Importance** ارزش مبحث در سرفصل است که نویسندهٔ محتوا ست می‌کند. **Recency** هم فاصلهٔ زمانی از آخرین برخورد با آن مبحث است تا مباحث رهاشده کم‌کم بالا بیایند و «فراموشی محض» رخ ندهد.

لایهٔ سوم – محدودیت مواجهه. آخرین فیلتر حفاظتی: سؤالی که در ۲۰ پاسخ اخیر دیده شده، از کاندیدها حذف می‌شود ($\text{exposure_window} = 20$). بدون این فیلتر، سیاست ارزش آموزشی ممکن است «عاشق» یک سؤال شود و هر جلسه تکرارش کند؛ نتیجه، حفظِ صورتِ سؤال و تورم مصنوعی نمره است. و یادت باشد: نوار سختی کل این انتخابات، از **سطح جاری نردبانِ آن دسته** (فصل بعد) می‌آید – دو موتور دست در دست هم کار می‌کنند.

```
"""انتخاب سؤال سه‌لایه - ناحیه هدف، ارزش آموزشی، محدودیت مواجهه"""

from elo import expected_success

TARGET = 0.70 # میانه ناحیه هدف
ZONE_LOW, ZONE_HIGH = 0.65, 0.75 # ناحیه هدف (پیوست الف)
EXPOSURE_WINDOW = 20 # پنجره مواجهه
W_WEAKNESS = 0.5 # وزن ضعف دسته
W_IMPORTANCE = 0.3 # وزن اهمیت میحت
W_RECENCY = 0.2 # وزن تازگی

def educational_value(weakness: float, importance: float,
                    recency: float) -> float:
    """ $V(q) = 0.50(1 + 0.2 \times \text{تازگی} + 0.3 \times \text{اهمیت} + 0.2 \times \text{ضعف})$ """
    return W_WEAKNESS * weakness + W_IMPORTANCE * importance \
        + W_RECENCY * recency
```

بخش ۲ – تابع اصلی انتخاب

`core/selector.py`

```
def select_question(r_student: float, candidates: list[dict],
                  recent_question_ids: list[int]) -> dict | None:
    """id, r_question, weakness, importance, recency بهترین سؤال بعدی؛ هر آیتم کا ندید شامل"""
    # لایه ۳ - محدودیت مواجهه: دیده‌شده‌های ۲۰ پاسخ اخیر حذف
    seen = set(recent_question_ids[-EXPOSURE_WINDOW:])
    pool = [c for c in candidates if c["id"] not in seen]
    if not pool:
        return None

    # لایه ۱ - ناحیه هدف (فال‌بک: نزدیک‌ترین به ۰.۷۰)
    for c in pool:
        c["e"] = expected_success(r_student, c["r_question"])
        zone = [c for c in pool if ZONE_LOW <= c["e"] <= ZONE_HIGH]
        if not zone:
            return min(pool, key=lambda c: abs(c["e"] - TARGET))

    # لایه ۲ - بیشترین ارزش آموزشی
    for c in zone:
        c["value"] = educational_value(
            c["weakness"], c["importance"], c["recency"])
    return max(zone, key=lambda c: c["value"])
```

فال‌بک لایه اول برای روزهای کم‌محتواست: اگر هیچ سؤالی در ناحیه نبود (بانک ۵۰ سؤالی هفته‌های اول)، به جای خطا، نزدیک‌ترین سؤال به ۰.۷۰ انتخاب می‌شود. تست‌های فصل ۶ سند توسعه، همین مسیر را پوشش می‌دهند.

مثال عددی گام به گام – دانش آموزی با توان ۱۲۳۰ و پنج کاندید

- ۱ برای هر کاندید E حساب می‌شود: سؤال $E = 0.870 \leftarrow 900$ خیلی آسان؛ سؤال $E = 0.500 \leftarrow 1230$ سکه‌ای؛ سؤال $E = 0.273 \leftarrow 1400$ خیلی سخت – هر سه از ناحیه خارج‌اند.
- ۲ دو کاندید داخل ناحیه ۶۵–۷۵٪ می‌مانند: سؤال ۱۱۱۰ با $E = 0.666$ و سؤال ۱۱۰۰ با $E = 0.679$.
- ۳ ارزش آموزشی: سؤال ۱۱۱۰ $\leftarrow 0.43 = 0.2 \times 0.2 + 0.3 \times 0.8 + 0.5 \times 0.3$ ؛ سؤال ۱۱۰۰ از دسته ضعیف‌تر $\leftarrow 0.77 = 0.2 \times 0.7 + 0.3 \times 0.6 + 0.5 \times 0.9$.
- ۴ برنده: سؤال ۱۱۰۰ – نه سخت‌ترین، نه آسان‌ترین؛ **مفیدترین** داخل ناحیه. (کل این جدول، خروجی واقعی تست است.)

دستگیره	مقدار فعلی	اگر عوضش کنی چه می‌شود؟
TARGET	0.70	پایین آوردنش (مثلاً ۰/۶) $\leftarrow$ تجربه سخت‌تر و استرس بیشتر؛ بالا بردنش $\leftarrow$ آسان‌گرایی و ملال
ZONE_LOW / HIGH	0.65 / 0.75	بازه باریک‌تر $\leftarrow$ سخت‌گیری بیشتر در انتخاب؛ بازه بازتر $\leftarrow$ تنوع بیشتر در بانک کم‌محتوا
W_WEAKNESS	0.5	بزرگ‌تر $\leftarrow$ سیستم عصبی‌تر دنبال ضعف‌ها می‌رود؛ ریسک یکنواخت‌شدن روی دسته‌های ضعیف
EXPOSURE_WINDOW	20	کوچک‌تر $\leftarrow$ تکرار زودتر (بد برای حفظ صورت سؤال)؛ بزرگ‌تر $\leftarrow$ بانک کم فشار می‌آید

چرا این سه فیلتر و نه Multi-Armed Bandit؟ الگوریتم‌های MAB (مثل UCB) مسئله اکتشاف/بهره‌برداری را بهینه حل می‌کنند، ولی سیاستشان به سختی قابل توضیح برای داور یا والد است و رفتار مرزی‌اش سخت دیباگ می‌شود. سه فیلتر آزمونک هر کدام یک جمله قابل دفاع دارند («ناحیه توسعه»، «اول ضعف‌ها»، «تکرار ممنوع») و پارامترهایشان مستقیم در آزمایش علمی حساسیت‌سنجی می‌شوند.

فصل ۴ – طرح جایگزین پویا

نردبان سطح D16 – لیگ‌های هر دسته

سه سطح برای هر دسته؛ سقوط با دو غلط پیاپی، ارتقای محتاطانه با دو درست پیاپی

یک عدد سراسری («مجموعاً چقدر بلدی؟») به دو سؤال حیاتی جواب نمی‌دهد: «کجای ریاضی‌ام ضعیفم؟» و «الان دقیقاً چه بخوانم؟». دانش‌آموزی با توان ۱۲۵۰ می‌تواند در «قاعده زنجیره‌ای» عالی و در «حد از چپ» ضعیف باشد. طراحی نخست برای این‌جا «موتور بدفهمی» بود: هر گزینه غلط به یک بدفهمی مفهومی مشخص مپ می‌شد. در بازبینی طراحی، پویا این مکانیزم را **وتو** کرد (D16): نویسندگی مپ برای هر گزینه، بار تولید محتوا را دو تا سه برابر می‌کرد. طرح جایگزین خودش، **نردبان سطح دسته‌ای** بود – ساده، شفاف و بدون هیچ ستون جدید دیتابیس – و همین‌جا پیاده‌سازی می‌شود.

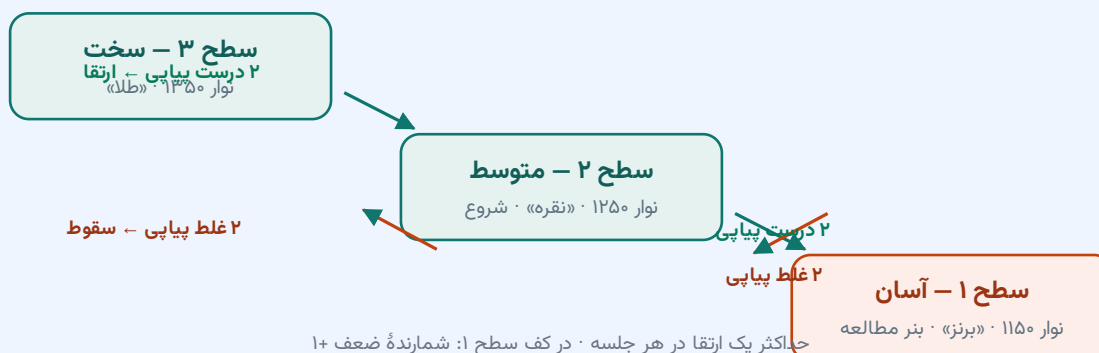
اگر یک بچه ۱۰ ساله بپرسد: «این لیگ برنز و نقره و طلا چیست؟»

بازی‌های آنلاین برای هر مهارت لیگ دارند: برنز، نقره، طلا. در آزمونک هم هر درس، لیگ‌های خودش را دارد – ممکن است در «معادله» لیگ طلا باشی و در «حالت‌های کلمه‌ای» هنوز لیگ برنز! دو بازی بباری و بباری (دو سؤال پشت‌سرهم غلط بزنی)؟ یک لیگ می‌افتید پایین. دو تا ببری؟ بالا می‌روی – ولی هر روز حداکثر یک پله، تا از یک جلسه خوش‌شانس هورا نکشی و لیگ‌ها بی‌اعتبار نشوند.

و اگر در پایین‌ترین لیگ هم مدام بباری؟ بازی سرت داد می‌زند: «دوست من، اول برو قواعدش را بخوان!» – همان **بنر مطالعه** که بالای صفحه می‌آید. این‌طوری سیستم تو را غرق سؤال‌های ناممکن نمی‌کند؛ به تو و کتاب درسیات برمی‌گرداند تا اول مطالعه کنی، بعد دوباره چالش را شروع کنی.

مفهوم – سه سطح، دو شمارنده، یک بنر

هر دسته (همان فیلد topic سؤال‌ها) برای هر دانش‌آموز یک سطح ۱ تا ۳ دارد که با سه رده سختی موجود سینک است: سطح ۱ ← نوار ۱۱۵۰، سطح ۲ ← ۱۲۵۰، سطح ۳ ← ۱۳۵۰. این سطح، «نوار سختی» را برای انتخاب سؤال فصل قبل تعیین می‌کند – پس هیچ ستون جدیدی در دیتابیس لازم نشد؛ فقط ردیف‌های دسته در ability_state.



شکل ۳ – نردبان سه‌سطحی هر دسته: سقوط با دو غلط پیاپی، ارتقای محتاطانه با دو درست پیاپی

چرا $n = 2$ و نه ۱ یا ۳؟ با $n=1$ هر غلط سطح را می‌اندازد – واکنشی ولی پرنوسان؛ $n=3$ محتاطانه ولی کند است. دو، تعادل سند است و اگر داده نشان دهد سطح‌ها پرش ناخواسته دارند، اول همین n بالا برده می‌شود. **چرا سقف ارتقا در جلسه؟** چون هدف «وضعیت واقعی» است نه «حس خوب لحظه‌ای»؛ ارتقای چندپله‌ای در یک جلسه خوش‌شانس، جلسه بعد را نابود می‌کند. **بنر مطالعه** هم دقیق تعریف شده: سطح ۱ و موفقیت زیر ۳۰٪ در ۵ پاسخ آخر همان دسته – یعنی «در پایین‌ترین لیگ هم به شدت می‌بازی». آستانه ۳۰٪ = یک از سه؛ بالابردنش بنرهای بیشتری می‌سازد و خطر «کری‌زدن سیستم» دارد.

```

"""سه لیگ برای هر دسته سؤال - (D16) نردبان سطح دسته‌ای"""

DROP_STREAK = 2          # n: غلط پیاپی لازم برای سقوط
RISE_STREAK = 2          # m: درست پیاپی لازم برای ارتقا
MIN_LEVEL, MAX_LEVEL = 1, 3
LEVEL_BANDS = {1: 1150, 2: 1250, 3: 1350} # نوار سختی هر سطح
STUDY_ALERT_SUCCESS = 0.30 # آستانه بنر مطالعه
ALERT_WINDOW = 5         # پنجره محاسبه موفقیت اخیر
START_LEVEL = 2          # شروع هر دسته از میانه (پیش‌فرض مهندسی)

class CategoryLadder:
    """وضعیت نردبان یک دسته برای یک دانش‌آموز"""

    def __init__(self, category: str) -> None:
        self.category = category
        self.level = START_LEVEL
        self.wrong_streak = 0      # شمارنده غلط پیاپی
        self.right_streak = 0     # شمارنده درست پیاپی
        self.weakness_counter = 0 # در کف سطح ۱: شمارش افت بیاثر
        self.promoted_this_session = False
        self.recent: list[bool] = [] # پنجره ۵ پاسخ آخر
        self.alert = False

    def difficulty_band(self) -> float:
        """این سطح - ورودی فیلتر انتخاب سؤال Elo نوار سختی"""
        return LEVEL_BANDS[self.level]

```

```
def record(self, correct: bool) -> dict:
    """ثبت یک پاسخ و بهروزرسانی سطح؛ خروجی: گزارش رویداد"""
    event = {"moved": False, "direction": None, "alert": False}
    self.recent.append(correct)
    self.recent = self.recent[-ALERT_WINDOW:]

    if correct:
        self.right_streak += 1
        self.wrong_streak = 0
        if (self.right_streak >= RISE_STREAK
            and self.level < MAX_LEVEL
            and not self.promoted_this_session):
            self.level += 1
            self.right_streak = 0
            self.promoted_this_session = True
            event.update(moved=True, direction="up")
    else:
        self.wrong_streak += 1
        self.right_streak = 0
        if self.wrong_streak >= DROP_STREAK:
            self.wrong_streak = 0
            if self.level > MIN_LEVEL:
                self.level -= 1
                self.promoted_this_session = False
                event.update(moved=True, direction="down")
            else:
                self.weakness_counter += 1

    if self.level == MIN_LEVEL and len(self.recent) == ALERT_WINDOW:
        rate = sum(self.recent) / len(self.recent)
        self.alert = rate < STUDY_ALERT_SUCCESS
        event["alert"] = self.alert
    return event
```

دو نکته ظریف: ① سقوط، شمارنده ارتقای جلسه را هم آزاد می‌کند (promoted_this_session = False) – یعنی در یک جلسه پرنوسان، سقوط و برگشت ممکن است ولی فقط یک بار. ② در کف سطح ۱، افت دیگر «بی‌اثر» است و به‌جایش شمارنده ضعف بالا می‌رود – این شمارنده همان ورودی هشدار مطالعه در گام ۴ سند است.

مثال عددی گام‌به‌گام – نردبان دسته «حد و مشتق» (خروجی واقعی تست)

- ۱ شروع: سطح ۲ (نقره). پاسخ‌ها: غلط، غلط ← **سقوط به سطح ۱**؛ شمارنده غلط پیاپی صفر می‌شود.
- ۲ درست، درست ← **ارتقا به سطح ۲**؛ سهمیه ارتقای این جلسه مصرف شد.
- ۳ درست (سومین درست پیاپی) ← سطح همان ۲ می‌ماند – ارتقای دوم همان جلسه ممنوع است.
- ۴ سناریوی کف: کسی در سطح ۱ با چهار غلط از پنج پاسخ آخر ← پنجمین پاسخ محاسبه می‌کند: موفقیت $30\% < 20\%$ ← **بنر مطالعه روشن**؛ اگر همزمان دو غلط پیاپی هم بیاید، به‌جای افت، شمارنده ضعف ۱+ می‌شود.

دستگیره	مقدار فعلی	اگر عوض کنی چه می‌شود؟
DROP_STREAK (n)	2	۱ ← هر غلط می‌اندازد (پرنوسان)؛ ۳ ← محتاطانه ولی کند؛ با پرش‌های ناخواسته، اول همین را بالا ببر
RISE_STREAK (m)	2	کاهشش حس پیشرفت سریع می‌دهد ولی سطح‌ها را شکننده می‌کند؛ از سقف «یک ارتقا در جلسه» جدا نگهش دار
STUDY_ALERT_SUCCESS	0.30	بالا بردنش (مثلاً ۰/۴) ← بنرهای بیشتر و ریسک «کری‌زدن سیستم»؛ پایین آوردنش ← بنرهای نادر و دیر
START_LEVEL	2	شروع از ۳ ← تجربه سنگین اولیه؛ شروع از ۱ ← تحقیر بچه‌های قوی؛ میانه بی‌طرف‌ترین است

چرا نردبان و نه BKT / DKT / KST؟ BKT (ردیابی دانش بیزی) برای هر مفهوم یک احتمال پنهان «یاد گرفته» نگه می‌دارد و DKT با شبکه عصبی آن را پیش می‌برد – هر دو قوی‌اند ولی به داده انبوه، تنظیم پارامتر و اعتماد ناشناخته نیاز دارند. نردبان با دو شمارنده و سه سطح، همین «ضعف محوری» را به دست می‌دهد و هر عددش را می‌توانی در جلسه دآوری از حفظ توضیح بدهی. و چون فرمت بدفهمی (ستون distractor_map و شیت misconceptions) حفظ شده، روزی که داده انباشت، BKT بدون تغییر اسکیمای روی همان داده فعال می‌شود.

فصل ۵

SM-2 – مرور فاصله‌دار

قبل از آن‌که فراموش کنی مرور کن: ۱ ← ۶ ← ۱۵ روز و بعد هر بار طولانی‌تر

Elo و نردبان درباره «فهم» تو حرف می‌زنند، ولی حافظه قانون خودش را دارد: هرچه یاد گرفته‌ای، اگر تمرین نشود، طبق یک منحنی ثابت محو می‌شود. SM-2 – الگوریتمی که پشت اپ‌هایی مثل Anki نشسته – این منحنی را با دو عدد مهار می‌کند: **آسانی نسبی EF** (این آیتم برای تو چقدر «چسبنده» است) و **شماره تکرار n**. بعد از هر پاسخ، فقط همین دو عدد به‌روز می‌شوند و فاصله مرور بعدی از دلشان بیرون می‌آید.

اگر یک بچه ۱۰ ساله بپرسد: «چرا بعضی سؤال‌ها برمی‌گردند سراغم؟»

مثل آبیاری گل است! اگر هر روز به گل آب بدهی، ریشه‌اش تنبل می‌شود و سطحی می‌ماند. اگر سه هفته فراموشش کنی، می‌خشکد. گل خورد هر روز آب نمی‌خواهد؛ دقیقاً لحظه‌ای که دارد تشنه می‌شود آب می‌خواهد. سؤال‌ها هم همین‌طورند: روز اول که یاد گرفتی، فردا دوباره می‌آیند؛ اگر خوب جوابشان بدهی، دفعه بعد شش روز بعد می‌آیند؛ باز هم خوب جواب بدهی، پانزده روز بعد؛ و هر بار فاصله‌ها طولانی‌تر می‌شود چون مغزت این‌ها را «محکم‌تر» نگه داشته.

حالا نکته باحالتش: گل‌ها با هم فرق دارند! بعضی سؤال‌ها «کاکتوس» اند – سخت‌گیر می‌شوی با آن‌ها و آب کم می‌خواهند؛ بعضی «بنفشه» اند – مدام تشنه‌اند. آن عدد EF همین است: برای هر سؤال، جداگانه می‌گویند چقدر ماندگار بوده. و اگر یک روز به یک سؤال کامل سربازنزدی؟ فاصله‌اش ریست می‌شود به فردا – مثل اینکه گل پژمیده و باید از اول بهش برسی.

مفهوم – کیفیت سه‌سیگنالی و فرمول درجه‌دو

کیفیت هر پاسخ، q نام دارد و بین ۰ تا ۵ است؛ در آزمونک از سه سیگنال ساخته می‌شود، نه فقط درست/غلط: ① درست – پایه: درست ← ۴، غلط ← ۲. ② اعتماد (D14) – «مطمئن بودم» یک امتیاز اضافه می‌کند (سقف ۵)؛ «حدس می‌زد» ولی درست، سقف q را در ۴ می‌بندد – چون حدس درست نباید حافظه واقعی قوی‌تری نشان بدهد. ③ تأخیر (D10) – پاسخ زیر ۱۵ ثانیه مشکوک به حدس رفلکسی است (سقف $q = 4$) و بالای ۷۰ ثانیه یعنی سخت به جواب رسیده‌ای (-۰/۵). سیگنال اعتماد فقط همین‌جا و در قرینه بدفهمی مصرف می‌شود و وارد نمره جلسه نمی‌شود – قاعده ثابت سند. دو عدد هر آیت با q به‌روز می‌شوند. اول آسانی نسبی؛ اگر $d = 5 - q$ را باز کنی، فرمول SM-2 شکل حساب‌شده‌اش را نشان می‌دهد:

$$EF' = EF + (0.1 - d \cdot 0.08 - d^2 \cdot 0.02) \quad d = 5 - q, \quad EF \geq 1.3$$

قانون آپدیت آسانی نسبی – جریمه درجه‌دو برای کیفیت بد؛ کف ۱/۳

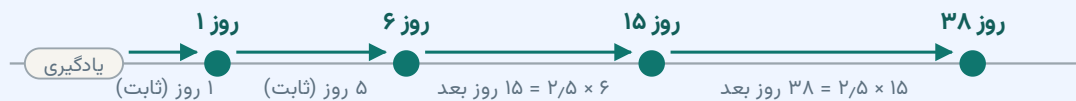
سه خاصیت از همین بازکردن بیرون می‌آید: در $q = 4$ ، $d = 1$ است پس $0.1 - 0.08 - 0.02 = 0$ و EF دست‌نخورده می‌ماند؛ در $q = 5$ فقط ۰/۱ اضافه می‌شود (رشد آهسته و صبور)؛ و هرچه q بدتر شود، جمله d^2 سنگین‌تر از جریمه خطی رشد می‌کند – کیفیت خیلی بد، سقوط EF را تندتر می‌کند. سپس فواصل:

$$q < 3 \Rightarrow \text{reset} \mid I(1) = 1, \quad I(2) = 6, \quad I(n) = I(n-1) \times EF$$

زنجیره مرور – یک غلط واقعی (q زیر ۳) زنجیره را به روز اول برمی‌گرداند

با $q = 5$ در هر مرور، فاصله‌ها با $EF = 2/5$ رشد می‌کنند: $1 \leftarrow 6 \leftarrow 15 \leftarrow 38 \leftarrow \dots$

یک غلط واقعی ($q = 2$) وسط راه؟ زنجیره ریست می‌شود و فردا از نو



شکل ۴ - زنجیره مرور یک آیتم سالم؛ هر پرش = ضرب در EF همان آیتم

یک ظرافت اجرایی: آستانه‌های تأخیر (۱۵ و ۷۰ ثانیه) پیش‌فرض سراسری‌اند و هر سؤال می‌تواند با فیلد `est_time_sec` خودش آن‌ها را `override` کند - سؤال محاسباتی سنگین حق دارد زمان بیشتری بخواهد. و «مطمئن اما غلط» (q پایین با اعتماد بالا) مهم‌ترین سیگنال انفرادی کل محصول است: همان قرینه بدفهمی که در گزارش جلسه ثبت می‌شود.

مهندسی - فایل `core/sm2.py` سه تابع خالص: کیفیت، آسانی، فاصله؛ حالت آیتم در دیتابیس می‌ماند 

"""آزمونک - مرور فاصله‌دار با کیفیت سه‌سیگنالی SM-2"""

```
EF_START = 2.5      # آسانی نسبی اولیه
EF_FLOOR = 1.3      # هرگز زیر آن نمی‌رود - EF کف
T_FAST = 15         # (q=4 سقف) ثانیه؛ زیر آن: حدس رفلکسی
T_SLOW = 70         # ثانیه؛ بالای آن: سخت به جواب رسیده (-0.5)
BASE_CORRECT = 4.0   # پایه برای پاسخ درست q
BASE_WRONG = 2.0     # پایه برای غلط (زیر ۳ - ریست فاصله) q
CONFIDENT_BONUS = 1.0 # مطمئن بودم «+۱»
GUESS_CAP = 4.0     # برای «حدس می‌زدم ولی درست q سقف»

def quality(correct: bool, confident: bool | None,
            elapsed_sec: float) -> float:
    """بین ۰ تا ۵ از سه سیگنال پاسخ q کیفیت"""
    q = BASE_CORRECT if correct else BASE_WRONG
    if confident is True:
        q = min(5.0, q + CONFIDENT_BONUS)
    elif confident is False and correct:
        q = min(q, GUESS_CAP) # حافظه واقعی قوی‌تر نیست
    if elapsed_sec < T_FAST:
        q = min(q, 4.0)      # پاسخ آنی مشکوک به حدس رفلکسی
    elif elapsed_sec > T_SLOW:
        q -= 0.5             # تلاش طولانی = یادگیری متزلزل‌تر
    return max(0.0, min(5.0, q))
```

```
def update_ef(ease_factor: float, q: float) -> float:
    """EF' = EF + 0.1 - d*0.08 - d^2*0.02 که d = 5 - q، ۱/۳ کف"""
    d = 5.0 - q
    new_ef = ease_factor + 0.1 - d * 0.08 - d * d * 0.02
    return max(EF_FLOOR, new_ef)

def next_interval(repetition: int, ease_factor: float, q: float,
                  previous_interval: int = 0) -> int:
    """یعنی ریست به ۱ روز q < 3 فاصله مرور بعدی برحسب روز؛"""
    if q < 3.0:
        return 1
    if repetition <= 1:
        return 1
    if repetition == 2:
        return 6
    return max(1, round(previous_interval * ease_factor))
```

جداسازی حالت از منطق: این فایل «بی‌حالت» است - repetition، ease_factor و previous_interval را از دیتابیس می‌گیرد و عدد برمی‌گرداند. به همین دلیل تست‌کردنش بی‌نیاز از سرور است و هسته (core) طبق قاعده پروژه هیچ I/O ندارد.

مثال عددی گام‌به‌گام - یک آیتم مرور در سه سناریو (خروجی واقعی تست)

- سناریوی درستِ مطمئن:** درست + «مطمئن بودم» + ۳۰ ثانیه $\leftarrow q = \min(5, 4+1) = 5$ EF: $2.5 \rightarrow 2.6$ (فقط +۱/۰) و زنجیره: $1 \rightarrow 6 \rightarrow 15 \rightarrow 38$.
- سناریوی حدسِ درستِ سریع:** درست + «حدس می‌زدم» + ۱۰ ثانیه $\leftarrow q = 4$ EF: $2.5 \rightarrow 2.5$ - ثابت! سیستم نه جریمه می‌کند نه جایزه؛ حدس درست رشد مرور نمی‌خرد.
- سناریوی غلطِ مطمئن:** غلط + «مطمئن بودم» + ۳۰ ثانیه $\leftarrow q = 2+1 = 3$ EF: $2.5 \rightarrow 2.36$ و چون $q \geq 3$ زنجیره حفظ می‌شود؛ اما همان غلط بدون اعتماد $\leftarrow q = 2 < 3$ فاصله بعدی **۱ روز** (ریست کامل) با $EF \rightarrow 2.18$.
- کف EF:** آیتمی با $EF=1/35$ که $q=0$ بگیرد $\leftarrow$ جریمه $d=5$ یعنی $0.55 = 0.8 - 1.35$ ، ولی کف اعمال می‌شود: $EF = 1.3$ - سادگی نسبی هرگز زیر کف نمی‌رود تا آیتم «غیرقابل‌بخشش» نشود.

دستگیره	مقدار فعلی	اگر عوض کنی چه می‌شود؟
T_FAST / T_SLOW	15 / 70	پس از MVP قرار است per-category با کالبراسیون داده واقعی شود (D10)؛ سؤال‌ها با est_time_sec می‌توانند override کنند
BASE_CORRECT / WRONG	4 / 2	q پایه؛ مرز جادویی ۳ جداکننده «ادامه زنجیره» از «ریست» است - این دو را فاصله نده

صفرش کنی ← دکمه اعتماد فقط سیگنال بدفهمی می‌دهد و در زمان‌بندی اثر ندارد	1.0	CONFIDENT_BONUS
پایین‌تر ← آیت‌های مشکل‌دار خیلی دیر برمی‌گردند و عملاً رها می‌شوند	1.3	EF_FLOOR

چرا SM-2 و نه Leitner / Pimsleur / FSRS / HLR؟ لایت‌نر فقط پنج جعبه ثابت دارد – برای «چه زمانی مرور» بسیار خشن است. Pimsleur برای واژگان صوتی طراحی شده و تعمیمش به ریاضی و علوم محدود است. FSRS و HLR (دوئولینگو) مدل‌های رگرسیونی/عصبی مدرنی‌اند که به داده واقعی هر کاربر برای کالیبراسیون نیاز دارند – در MVP داده نداریم. SM-2 نقطه طلایی است: بدون نیاز به داده شروع می‌شود، از روز اول منطقی رفتار می‌کند و بعداً با داده جمع‌شده می‌تواند به FSRS ارتقا یابد.

فصل ۶

پیش‌بینی نمره – شفاف‌سازی راز

نمره وزن‌دار با سختی (D13) + رگرسیون نمایی $\lambda = 0.85$

آخرین پازل داشبورد: عددی که به دانش‌آموز می‌گوید «اگر همین‌طور ادامه بدهی، جلسه بعد حدوداً این‌قدر می‌گیری». این عدد دو وظیفه دارد: انگیزه‌سازی صادقانه و روایت‌سازی برای داوری («سیستم ما روند یادگیری را پیش‌بینی می‌کند»). ولی پیش‌بینی روی نمره خام، فریب است – ۷۰٪ از سؤال‌های آسان با ۷۰٪ از سؤال‌های سخت یکی نیستند. پس اول نمره جلسه باید «سخت‌گیرانه» تعریف شود (D13) و بعد روندش مدل شود.

اگر یک بچه ۱۰ ساله بپرسد: «چطور نمره جلسه بعدی من را حدس می‌زند؟»

معلم‌تان را تصور کن که برگه‌های ده‌تایی‌ات را نگه داشته و می‌گوید: «این هفته‌ها همیشه بهتر شدی؛ آخرین برگه‌ها هم خوب بودند – به نظرم جلسه بعد حدود ۷۵ می‌گیری». آزمونک هم همین کار را می‌کند ولی منصفانه‌تر: برگه‌های **آخر** را بیشتر جدی می‌گیرد (شاید هفته اولت بد بود چون تازه بودی!) و برگه‌های خیلی قدیمی را کم‌کم فراموش می‌کند.

و نکته مهم انصاف: سؤال سخت درست‌زده بیشتر نمره دارد! اگر کسی فقط سؤال‌های آسان را بزند، نمره‌اش همان‌قدر بالا نمی‌رود – مثل این است که معلم بگوید «ده تا درست زدی ولی همه‌شان از بخش‌های آسان بود، پس نمره کامل نمی‌گیرم». همان هفت‌تای درست، اگر از سؤال‌های سخت باشد، نمره خیلی بهتری می‌آورد!

مفهوم – اول نمره منصفانه، بعد روند وزن‌دار

گام اول – نمره جلسه (D13). هر سؤال، وزنی برابر با رده سختی‌اش می‌گیرد: آسان (۱۱۵۰) $\times 1$ ، متوسط (۱۲۵۰) $\times 1.5$ ، سخت (۱۳۵۰) $\times 2$. نمره جلسه یعنی سهم وزنی درست‌ها از کل وزن:

$$\text{Score} = \sum (w_i \times s_i) \div \sum w_i \times 100$$

نمره وزن‌دار جلسه – وزن تابع سختی Elo سؤال؛ سیگنال اعتماد عمداً این‌جا نیست

گام دوم – رگرسیون روند. پیش‌بینی، میانگین وزن‌دارِ نماییِ نمره‌های جلسات است: جلسه آخر وزن $\lambda^0 = 1$ ، یکی مانده به آخر $\lambda = 0.85$ ، دوتا مانده $\lambda^2 \approx 0.72$ و به همین ترتیب. $\lambda = 0.85$ یعنی «حافظه بلندمدت با احتیاط»: جلسات اخیر حاکم‌اند ولی گذشته هنوز صدا دارد. پیش از ۸ جلسه، پیش‌بینی اصلاً نمایش داده نمی‌شود – عددِ کم‌پایه اعتماد می‌سوزاند. توان R_s هم جداگانه در نمودار روند رسم می‌شود (D15): نمره برای دانش‌آموز، R_s برای روایت دفاع.

مهندسی – فایل `core/predict.py` سه تابع: وزن سختی، نمره جلسه، پیش‌بینی روند 

```
""" λ رگرسیون نمایی + D13 پیش‌بینی نمره – نمره وزن‌دار """

LAMBDA = 0.85          # وزن نمایی جلسات اخیر
MIN_SESSIONS = 8       # حداقل جلسه برای نمایش پیش‌بینی
TIER_WEIGHTS = {1150: 1.0, 1250: 1.5, 1350: 2.0}

def question_weight(r_question: float) -> float:
    """نزدیک‌ترین رده ۱۱۵۰/۱۲۵۰/۱۳۵۰ - وزن سؤال از سختی"""
    nearest_tier = min(TIER_WEIGHTS, key=lambda t: abs(t - r_question))
    return TIER_WEIGHTS[nearest_tier]

def session_score(answers: list[tuple[bool, float]]) -> float:
    """۱۰۰ × (وزن) ÷ (وزن × درستی): نمره وزن‌دار"""
    total_weight = 0.0
    earned_weight = 0.0
    for correct, r_question in answers:
        w = question_weight(r_question)
        total_weight += w
        if correct:
            earned_weight += w
    if total_weight == 0:
        return 0.0
    return earned_weight / total_weight * 100.0

def predict_next_score(session_scores: list[float],
                       lam: float = LAMBDA) -> dict:
    """یعنی آخرین جلسه i=۰ میانگین نمایی وزن‌دار!"""
    if len(session_scores) < MIN_SESSIONS:
        return {"available": False,
                "reason": f"جلسه لازم است {MIN_SESSIONS} حداقل"}
    weights = [lam ** i for i in range(len(session_scores))]
    forecast = (sum(w * s for w, s in zip(weights, session_scores))
                / sum(weights))
    naive_mean = sum(session_scores) / len(session_scores)
    return {"available": True, "forecast": round(forecast, 1),
            "naive_mean": round(naive_mean, 1),
            "direction": "up" if forecast >= naive_mean else "down"}
```

مثال عددی گام به گام – همان مثال G1 سند (خروجی واقعی تست)

- ۱ جلسه‌ای با وزن‌های (آسان 1×1 ، متوسط 3×1.5 ، سخت 6×2) $\leftarrow \Sigma w = 12.5$
- ۲ هفت درست «۴ آسان + ۲ متوسط + ۱ سخت» $\leftarrow (4 \times 1 + 2 \times 1.5 + 1 \times 2) \div 12.5 \times 100 = 9 \div 72 = 12.5 \times 100$
- ۳ همان هفت درست اما اگر همگی از سؤال‌های آسان بود $\leftarrow 56 = 7 \times 1 \div 12.5 \times 100$ - سختی واقعاً دیده می‌شود؛ دقیقاً چیزی که نمره خام نمی‌فهمد.
- ۴ پیش‌بینی با نمره‌های آزمایشی [52, 58, 60, 65, 63, 70, 68, 74] $\leftarrow$ میانگین نمایی 61.4 در برابر میانگین ساده 63.8؛ چون جلسات اخیر گاهی پایین‌ترند، پیش‌بینی حساب شده محافظه‌کارتر از میانگین بی‌وزن است.

چرا میانگین نمایی و نه رگرسیون خطی / LSTM؟ رگرسیون خطی روی نمره‌های ناپایدار ۸ جلسه‌ای اول، به نوسان‌ها بیش‌ازحد واکنش نشان می‌دهد و LSTM به هزاران داده نیاز دارد. میانگین نمایی با یک پارامتر (λ) رفتار «حافظه اخیرگرا» را دقیق کنترل می‌کند، در آزمایش علمی قابل تفسیر است و اگر بد پیش‌بینی کرد، معلوم است کدام پارامتر را بچرخانی.

فصل ۷

قواعد اجرا – قانون‌های بازی

جلسه، دکمه اعتماد و مرزهای گزارش: چهار قاعده‌ای که تجربه کاربری را منصف نگه می‌دارد

الگوریتم‌های پنج فصل قبل، موتورند؛ اما یک موتور بدون ترافیک چراغ، تصادف می‌سازد. این فصل کوتاه‌تر است چون «الگوریتم» ندارد – چند قاعده اجرایی است که سند اصلی و تصمیم‌های جلسات تعیین‌شان کرده‌اند و هرکدام دلیل حساب شده دارند. قاعده‌ها را بفهمی، همه چیز بقیه محصول هم جا می‌افتد.

👤 اگر یک بچه ۱۰ ساله بپرسد: «قانون‌های کلاش چیست؟»

هر بازی دقیقاً **۱۰ مرحله** دارد – نه کم، نه زیاد؛ مثل مسابقه‌ای که همه مسافتشان یکی است تا امتیازها قابل مقایسه باشند. بعد از زدن جواب و **قبل از اینکه سیستم جواب را نشان بدهد**، دو دکمه بزرگ می‌آید: «مطمئن بودم» یا «حدس می‌زدم» – این لحظه تنها لحظه‌ای است که حرف راست می‌زنی؛ بعد از دیدن جواب، همه ادعا می‌کنند مطمئن بودیم!

و بین دو جلسه باید دست کم **۲ ساعت فاصله** باشد؛ چون هدف «تقویت واقعی» است نه «رونویسی از حافظه گرم». اگه همین الان چیزی را بخوانی و پنج دقیقه بعد آزمونش بدهی، انگار از خودت سؤال نپرسیده‌ای، از حافظه موقت پرسیده‌ای!

💡 مفهوم – چرا دقیقاً این‌طوری؟

چرا ۱۰ سؤال ثابت (D12)؟ نمره جلسه (D13) ورودی رگرسیون و نمودار روند است؛ اگر جلسات طول متفاوت داشته باشند، نمره‌ها هم‌وزن نیستند و روند درهم می‌ریزد. طول ثابت یعنی هر نقطه نمودار دقیقاً هم‌معنا با بقیه. فقط جلسه جایابی مستثنی است (۳ تا ۵ سؤال) چون ذاتش فرق دارد.

چرا دکمه اعتماد قبل از بازخورد (D14)؟ سه گزینه بررسی شد و «بعد از انتخاب، قبل از بازخورد» انتخاب شد: داده یکنواخت برای همه سؤال‌ها می‌دهد، سوگیری گزارش «بعد از دیدن جواب» را می‌بندد و فقط ۱ لمس هزینه دارد. این سیگنال دو مصرف دارد: ورودی **q** در SM-2 و «قرینه بدفهمی» (مطمئن اما غلط = قوی‌ترین زنگ خطر). **چرا اعتماد وارد نمره جلسه نمی‌شود؟** قاعده ثابت سند: نمره سنجش «توان» است و اعتماد سنجش «کیفیت حافظه»؛ خلط این دو، نمره را قابل دستکاری می‌کند – بچه می‌فهمد اگر «حدس می‌زد» بزند نمره‌اش می‌افتد و گزارش را شروع می‌کند به بازی دادن.

دو قاعده پشتیبان. ① **هشدار نوسان سطح** (گام ۴/۴): اگر سطح یک دسته در جلسه‌ای بیش از یک بار جابه‌جا شود، سیستم آن دسته را در گزارش جلسه با برچسب «نوسان» نشان می‌دهد – جانشین دکمه اعتراض طرح اول. ② **Snapshot پارامترها (G7)**: در هر پاسخ، مقادیر همه دستگیره‌ها همان لحظه ثبت می‌شوند؛ هر نتیجه تجربی بعداً با همان تنظیمات دقیق قابل بازتولید است – نعمت داوری علمی.

قاعده	مقدار / محل	پشت این انتخاب چه چیزی است؟
طول جلسه (D12)	سؤال ثابت ۱۰	هم‌وزن‌شدن نمره‌ها برای رگرسیون و نمودار روند
فاصله دو جلسه (D12)	دقیقه ۱۲۰ $\geq$	جلوگیری از آزمون حافظه گرم؛ مبنای قاعده دو شاهد پارک‌شده
دکمه اعتماد (D14)	بعد از گزینه، قبل از بازخورد	داده یکنواخت، سوگیری پس‌آگاهی بسته، هزینه: ۱+ لمس
مسیر سیگنال اعتماد	و قرینه بدفهمی q فقط	نمره جلسه دست‌نخورده می‌ماند و غیرقابل دستکاری است
هشدار نوسان سطح	برچسب در گزارش جلسه	شفافیت به‌جای دکمه اعتراض حذف‌شده
Snapshot پارامترها (G7)	در هر پاسخ params_snapshot	بازتولید کامل هر نتیجه؛ محافظ دفاع علمی

قاعده طلایی پشت همه این‌ها: هیچ عددی که به دانش‌آموز نشان داده می‌شود، قابل بازی‌دادن نیست. نمره از سختی واقعی می‌آید، پیش‌بینی از جلسات واقعی، سطح از پیروزی/شکست واقعی و اعتماد هم قبل از هرگونه بازخورد گرفته می‌شود. صادق‌بودن سیستم، شرط بقای انگیزه است.

همه چیز در یک نگاه

چرخه کامل + جدول ۱۶ تصمیم مهندسی پروژه

کل آزمونک یک حلقه است: **جایابی** عدد اولیه را می‌سازد؛ **نردبان** برای هر دسته نوار سختی تعیین می‌کند؛ **انتخاب سؤال** داخل آن نوار، سؤال ناحیه ۶۵-۷۵٪ با بیشترین ارزش آموزشی را برمی‌گزیند؛ پاسخ تو با **Elo** دوطرفه تسویه می‌شود؛ **SM-2** سؤال را برای مرور آینده زمان‌بندی می‌کند؛ و پایان جلسه، **نمره وزن دار و پیش‌بینی** تولید می‌شود. جدول زیر، شانزده تصمیمی است که این چرخه را شکل داده‌اند – تاریخ دفاع جشنواره، همین جدول است.

#	موضوع	انتخاب نهایی
D1	توان اولیه	جلسه جایابی ۴ سؤالی با سختی پراکنده و K ثابت ۳۲
D2	آقلاین PWA	فعلاً آنلاین؛ سرویس‌ورکر فقط برای نصب‌پذیری و کش پوسته
D3	احراز هویت	OTP پیامکی، کد ۵ رقمی، توکن کوتاه‌عمر
D4	فونت فارسی	وزیرمتن با subset و unicode-range؛ بودجه ۲۰۰ کیلوبایت صفحه
D5	زبان مخزن	دوزبانه: کد انگلیسی، مستندات فارسی + انگلیسی، UI فارسی
D6	بانک سؤال	Google Sheets با اسکیمای سفت + اسنپ‌شات YAML/JSON در مخزن
D7	میزبانی مخزن	GitHub خصوصی از روز اول ← عمومی در پایان فاز ۲
D8	ترتیب کار	اول بستن سؤالات باز، بعد کد از گام ۰
D9	سرویس پیامک	تعویق؛ provider=console در کل توسعه؛ اتصال پیش از آزمایش میدانی
D10	آستانه‌های تأخیر SM-2	$t_{fast} = 15$ ثانیه، $t_{slow} = 70$ ثانیه
D11	هویت و داده پژوهش	تفکیک کامل؛ جدول‌های identity و otp_codes مستقل
D12	تعریف جلسه	۱۰ سؤال ثابت؛ فاصله حداقلی دو جلسه = ۲ ساعت
D13	نمره جلسه	وزن دار با سختی: $\sum(\text{وزن} \times \text{درستی}) \div \sum(\text{وزن}) \times 100$
D14	دکمه اعتماد	بعد از انتخاب گزینه، قبل از بازخورد؛ دو دکمه بزرگ
D15	نمودار روند	دو خط: نمره جلسه + توان R_S (SVG دستی)
D16	لایه بدفهمی	پارک موتور مپ؛ جانشین: نردبان سطح دسته‌ای (فصل ۴ همین کتاب)

مسیر پیشنهادی برای آمادگی داوری: ① از این کتاب شکل ۱ و فصل ۱ را بلند توضیح بده (الگوریتم محوری). ② فصل ۴ را با افتخار تعریف کن – «طراحی من با وتوی خودم بهتر شد» بهترین روایت مهندسی است. ③ اگر داور پرسید «چرا IRT نزدی؟»، باکس‌های پایان فصل‌ها آماده پاسخ‌اند. ④ و اگر خواست عمیق‌تر برود، نسخه تخصصی ۳۱ صفحه‌ای کنار همین سند است.

دستگیره‌های قابل تنظیم – جدول کامل پارامترها

هر عددی که می‌خواهی آزمونک «دیگری» باشد، از این‌جا چرخانده می‌شود (مطابق پیوست الف سند توسعه v1.3)

این جدول همه پارامترهای تصمیم‌گیر سیستم را یک‌جا دارد. همه آن‌ها در params_snapshot هر پاسخ ثبت می‌شوند (G7) و در آزمایش علمی، حساسیت‌سنجی می‌شوند. همین مقادیر، به‌صورت ثابت‌های بالای هر فایل در core/ هم آمده‌اند – تغییر بدهی، رفتار سیستم عوض می‌شود و هیچ جای دیگری از کد دست‌نخورده می‌ماند؛ این معنای «دستگیره» است.

پارامتر	مقدار اولیه	توضیح
دانش‌آموز (شروع) K	32	پس از ۵۰ پاسخ ← ۲۴؛ پس از ۱۵۰ پاسخ ← ۱۶
سؤال K	8	پس از ۴۰ پاسخ ← ۴؛ رأی هر پاسخ درباره سؤال کم‌ارزش‌تر است
ناحیه هدف انتخاب	0.65 – 0.75	حساسیت‌سنجی در آزمایش علمی
مرجع انتخاب	0.70	کمینه فاصله E از میانه ناحیه هدف
پنجره مواجهه	20	سؤال دیده‌شده در ۲۰ پاسخ اخیر، از کاندیدها حذف (G8)
افت سطح نردبان n	2	غلط پیاپی در یک دسته؛ کف سطح = ۱؛ در کف: شمارنده ضعف ۱+
ارتقای نردبان m	2	درست پیاپی؛ سقف سطح = ۳؛ حداکثر یک ارتقا در هر جلسه
آستانه هشدار مطالعه	0.30	سطح ۱ + موفقیت زیر ۳۰٪ در ۵ پاسخ آخر دسته
سطوح نردبان	1150 / 1250 / 1350	سه رده سختی؛ سطح جاری، نوار انتخاب سؤال را تعیین می‌کند
SM-2 در EF کف	1.3	سادگی نسبی هرگز زیر کف نمی‌رود
شروع EF	2.5	آسانی نسبی اولیه هر آیت‌م مرور
t_fast / t_slow	15s / 70s	آستانه‌های تأخیر D10؛ پس از MVP: per-category با کالبراسیون
فاصله حداقلی دو جلسه	دقیقه 120	مبنای قاعده دو شاهد (D12)
طول جلسه	سؤال ۱۰	جلسه جایابی مستثنی (۳-۵ سؤال)
تعریف نمره جلسه	D13 وزن‌دار	$\Sigma(\text{وزن} \times \text{درستی}) \div \Sigma(\text{وزن}) \times ۱۰۰$ ؛ وزن‌های رده: ۲ / ۱/۵ / ۱
وزن روند پیش‌بینی	$\lambda = 0.85$	وزن نمایی جلسات اخیر در رگرسیون
حداقل جلسه برای پیش‌بینی	8	پیش از آن، عدد پیش‌بینی نمایش داده نمی‌شود
تعداد سؤال جایابی	(بازر ۳-۵) 4	سختی پراکنده روی سه رده؛ K ثابت ۳۲ (D1)

پارامتر	مقدار اولیه	توضیح
بودجه حجم صفحه آزمون	200 KB	Lighthouse CI از گام ۳ محافظت می‌کند
MVP سهمیه بانک سؤال	۵۰ ← ۱۵۰	هدف هفته ۵ / هفته ۸
اندازه گروه‌های آزمایش	۲۰ + ۲۰	تصادفی‌سازی بلوکه‌شده؛ آزمایش علمی سند

پیوست ب

واژه‌نامه و اجرای ماثول‌ها

هر اصطلاح این کتاب، یک خطی؛ و راه انداختن کد در سه دستور

واژه‌نامه – فارسی ↔ English

فارسی	English	یعنی
توان	R_s (ability)	امتیاز Elo دانش‌آموز؛ شروع از ۱۲۵۰ با جلسه جایابی
سختی	R_q (difficulty)	امتیاز Elo سؤال؛ با رأی پاسخ‌دهندگان اصلاح می‌شود
انتظار	E (expected)	شانس درست‌زدن؛ از فرمول لجستیک با ثابت ۴۰۰
ضریب یادگیری	K (factor)	سرعت حرکت امتیاز؛ با تجربه آرام می‌گیرد
جایابی	Placement	جلسه ۴ سؤالی تعیین توان اولیه (D1)
ناحیه هدف	Target zone	بازه شانس ۶۵–۷۵٪ برای سؤال بعدی
ارزش آموزشی	V(q)	امتیاز «مفید بودن»: ضعف + اهمیت + تازگی
پنجره مواجهه	Exposure window	سؤال دیده‌شده در ۲۰ پاسخ اخیر ممنوع است
دسته	Category	مبحث ریز سؤال؛ فیلد topic در بانک سؤال
نردبان / سطح	Ladder / Level	سه لیگ هر دسته: ۱۱۵۰، ۱۲۵۰، ۱۳۵۰ (D16)
بهر مطالعه	Study alert	هشدار «مطالعه کن»: سطح ۱ + موفقیت زیر ۳۰٪
مرور فاصله‌دار	Spaced repetition	برگشت سؤال درست قبل از فراموشی؛ موتور SM-2
آسانی نسبی	EF (ease factor)	ضریب رشد فاصله هر آیت؛ شروع ۲/۵، کف ۱/۳
کیفیت پاسخ	q (quality)	عدد ۰ تا ۵ از سه سیگنال: درستی، اعتماد، تأخیر
فاصله مرور	I(n) (interval)	روزهای تا مرور بعد: ۱ ← ۶ ← EF × (n-1)
نمره وزن‌دار	Weighted score	نمره جلسه با وزن سختی سؤال‌ها (D13)
رگرسیون نمایی	λ-weighted trend	پیش‌بینی با حافظه اخیرگرا؛ $\lambda = ۰٫۸۵$
اسنپ‌شات پارامترها	params_snapshot	ثبت مقادیر همه دستگیره‌ها در هر پاسخ (G7)

اجرای مازول‌ها – سه دستور

فایل‌های core/ کنار همین PDF، پایتون خالص‌اند (نسخه ۳/۱۰ به بالا، بدون هیچ کتابخانه‌ای) و هر عددی که در این کتاب دیدی از همین‌ها بیرون آمده. برای امتحان:

```
# ۱) را همین حالا بازتولید کن Elo تست طلایی (۱):
cd azmoonak-algorithms-3layer/core
python3 -c "from elo import apply_result; print(apply_result(1200, 1350, True))"
# خروجی: (0.2966, 1344.37, 1222.51) ← همان ۱۲۲۲٫۵ و ۱۳۴۴٫۳ کتاب

# ۲) یک جلسه جایابی کامل (۲):
python3 -c "from placement import run_placement; print(run_placement([True, False, True, True]))"

# ۳) همه تست‌های هر شش مازول (همان اعداد مثال‌های کتاب) (۳):
python3 /path/to/test_core_modules.py # خروجی: ALL OK ✓
```

پیشنهاد تمرین: در elo.py مقدار D_SCALE را به ۲۰۰ تغییر بده و تست را دوباره اجرا کن؛ جهش امتیاز دانش‌آموز چه قدر می‌شود؟ بعد DROP_STREAK در leveling.py را ۱ کن و ببین نردبان چه قدر عصبی می‌شود. بازی کردن با دستگیره‌ها، بهترین درس است.

حالا سه بار فهمیدی؛ یک بار برای قصه، یک بار برای مغز، یک بار برای کد.

هر عدد این کتاب با کد واقعی تست شده و هر فایل core/ همین حالا اجرا می‌شود.
دستگیره‌ها دست خودت است — بچرخان‌شان، بشکن‌شان، دوباره بساز.
این دقیقاً همان زبانی است که در جلسهٔ داوری جشنواره، تو را از «یک اپ آزمون» جدا می‌کند.

کد

مفهوم

قصه